

比赛注意事项

考试前

一、准备物品

准考证、身份证、笔。**禁止**携带 U 盘、MP3、计算器、手机等任何与存储、计算、通信有关的电子设备；禁止携带一切书籍和其他无关物品；一经发现按作弊处理。草稿纸考场会发，无需自己准备。

二、保持好的心态

考试前不要过于紧张，可以回忆一下以前考试常用的技巧，易错点和“骗分方法等”

三、赛前准备时间

比赛提前 30 分钟进场，熟悉考场环境。

利用熟悉考场时间写下重要且易错内容，如：文件操作命令等。

试题解压密码会影响一个人的心情，一定要一次输对，注意大小写，不要边输入边检查，要对自己有自信。

四、提交代码前注意事项

- 1、检查内容：所有代码的文件名（拓展名为.cpp）要小写且不能写错；freopen（）里的文件名要小写且不能写错；中间计算结果是否超 int；long long 类型输入（scanf）、输出（printf）是否是用了“%lld”；调试语句是否删除或屏蔽；分块是否有错；是否需要高精度；是否交错文件。
- 2、源代码务必存放在目录路径 D:\noip 下。（注意：noip 下不要建立子文件夹）。
- 3、在磁盘根目录下 Dev-C++可能会无法调试，建议在 D:\noip 目录下进行代码编

写与调试。

4、不能使用的变量名：dfs、next、search、index、pow（幂函数）、min、max、y1、x0、link、map、pipe，ws

5、可以使用：#include<bits/stdc++.h>

考试中

一、答题策略

过程： 读题->分析题意->找出算法->编写程序->手动测试：样例、自测数据->文件测试：与样例对比

几个注意点：

(1)、浏览试题，阅读并分析

(2)、先易后难，每完成一题要调试好、保存好。

(3)、容易题要保证测试数据全过，难的问题尽可能取得一些边界分数。

(4)、阅读要小心，分析要全面，可借助图示等方法理解题意。

注意数组是否越界!!! 全局变量与局部变量尽量不相同。递归有层次限制，最多层数与程序大小、电脑配置有关。考虑特殊情况和极限情况。注意经常保存文件!!!!

1.审题：

这一点非常重要，一旦审题错误或者理解错误就可能造成你花很多时间写出来的程序 WA。

如果没有思路，可以尝试着多读几次题目。

2.分段的思想

任何题不管有没有思路，先写出最简单的暴力做法（一眼能够看出怎么做的题除

外)，再逐步思考正解。当想不出来正解的时候，根据数据范围分段写暴力、骗分。

可用 namespace 封装几个分段的做法，避免变量命名重复以引起冲突。

遇到高精度的题目，先写低精度，再写高精度，并用低精度做对拍。

3.考虑严谨：

这一点也相当重要，如果考虑不严谨就可能被特殊数据卡分[0,100]而特殊数据往往分为极端数据和特殊数据。极端数据会按数据最大范围来，所以要注意空间是否足够，int 是否会溢出；数组的大小是否合适。特殊数据往往是具有特殊情况或性质的数据往往需要特判。

4.时空复杂度

通常 1s 运行时间最放心的时间复杂度为 $1e6$ ，时间复杂度为 $1e7$ 时尽量降低代码常数，时间复杂度为 $1e8$ 时尽量不冒险。

查询空间复杂度可用 sizeof 函数

sizeof(T)返回存储一个类型 T 的对象所需要的字节数。

例如：

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
    printf("%zu ",sizeof(char));
```

```
    printf("%zu",sizeof(int));
```

```
    printf("%zu ",sizeof(float));
```

```
    printf("%zu ", sizeof(double));
```

```
    return 0;

}
```

结果：1 4 4 8

sizeof(数组)可得数组所占的字节数

大数组开全局变量。

5.freopen:

文件输入输出是至关重要的一部分，一旦写错一点，就会丢掉该题的全部分数

先按照平时流程完成代码，确认代码无误且没有输出多余的调试语句后，再加上

文件输入输出以及头文件，再次编译，编译无误时则可认为完成该题。

在 return 0;前加上 fclose(stdin);fclose(stdout);关闭输出流

6.检查:

非常重要，需要综合注意事项中的 2,3,4,5,7，以及考试技巧中的的对拍

还有注意检查你的程序是否在考试指定文件夹里面，名称是否正确

尤其是.cpp/.in/.out 的名称

注意看清题目数据范围，数组范围开小了会导致数组越界从而 RE，数组范围开

大了导致 MLE 检查每个程序代码中两行 freopen 语句是否存在，是否被注释。

7.头文件

万能头文件:#include<bits/stdc++.h>

若在考前不能确定是否能用万能头文件，请使用以下的常用头文件：

```
#include<iostream>
```

```
#include<iomanip>
```

```
#include<cmath>
```

```
#include<cstdio>

#include<cstring>

#include<algorithm>

#include<queue>

#include<list>

#include<set>

#include<stack>

#include<vector>

#include<deque>

#include<map>

#include<stdlib>
```

由于评测环境是在 linux 平台下，切记不要使用 linux 平台没有的头文件，如 `#include<windows.h>`。

一些英文变量的命名可能与 linux 平台下的关键字冲突，但在 windows 平台下可以编译通过，如 `time` 等，使用英文变量名最好首字母大写。

考试技巧

1.时间复杂度的选择

根据每道题数据范围选择适当时间复杂度的算法

2.利用 Dev C++

如果你的考试环境使用 Dev C++那么注意以下：

a.在工具->编译选项 中将代码警告选项调为显示最多警告信息,帮助你尽可能多的发现隐性错误

3.巧用 cmd (选学)

说明: 这个是对拍程序的基础

a.启动目标在当前文件夹的 cmd

b.使用 cmd 对比数据

标准数据太长的时候, 肉眼观察肯定是一件恼火的事情, 这里我们用 `fc(file compare?)`

格式: `fc ans1.txt ans2.txt`

`ans1.txt ans2.txt` 代表两个不同的需要对比的文件, 但是注意 `fc` 比较时不会忽略行末空格

注意 `ans1.txt ans2.txt` 必须在 `cmd` 的目标文件夹里面

c.启动你的程序

格式: `code.exe < in.txt > out.txt`

表示启动名叫 `code` 的 `exe` 程序并输入 `in.txt` 的内容, 再将输出的内容保存到 `out.txt` 中, 这样你不需要用 `freopen`

注意 `code.exe` 必须在 `cmd` 的目标文件夹里面

4.对拍 (提高组)

.cpp 的写法

```
#include<iostream>
```

```
#include<windows.h>
```

```
using namespace std;
```

```

int main()
{
    //int t=200;

    while(1)
    {
        t--;

        system("data.exe > data.in");//data.exe 生成 data.in

        system("biaoda.exe < data.in > biaoda.out");//程序 1 生成.out 文件

        system("test.exe < data.in > test.out");//程序 2 生成.out 文件

        if(system("fc test.out biaoda.out"))    break;//两个.out 文件进行比较

    }

    if(t==0) cout<<"no error"<<endl;

    else cout<<"error"<<endl;//可在输出 error 后加入 color 语句使得对拍出错更加醒目。

    //system("pause");

    return 0;
}

```

.bat 文件的写法

@echo off

:Loop

MAKEDARA.exe>1.in

A.exe<1.in>1.out

B.exe<1.in>2.out

fc 1.out 2.out

if errorlevel 0 goto loop

pause

5.如何生成测试数据

前言：如何生成随机数

头文件：<cstdlib>

产生随机数：

① 0~32767 的随机数：rand()

② 粗略地控制范围：rand()%范围

注意，这种方法产生的随机数的分布是不均匀的。

③ 精确地控制范围：(double)rand()/RAND_MAX*范围

④ 控制在[a, b)之间：a + (int) ((double)rand()/RAND_MAX*(b-a))

初始化随机数种子：

① `srand(数字)`：初始化随机数种子。

② 注意，这条语句在程序开头使用，并且最多用一次。同一程序、同一平台，`srand` 中的参数相等，用 `rand()`产生的随机数序列相同。

③ 使随机数更加随机：引用 `<ctime>`，然后这样初始化随机数种子 `srand(time(NULL))`。

不要在循环中使用这条语句（例如批量产生随机数据），因为 `time` 只精确到秒。

一、排列

1、问题：给定 N ，在 $O(n)$ 随机出 $1—N$ 的一个排列

```
#include <algorithm>
```

```
int a[...]=...;
```

```
next_permutation(a+1,a+n+1)
```

2、随机排序

时间复杂度： $O(N)$

代码：

```
int t[...];
```

```
void makePerm0(void)
```

```
{
```

```
    for (int i=1;i<=n;i++) t[i]=i;
```

```
    random_shuffle(t+1,t+n+1);  
}
```

TL 中的函数 `random_shuffle()` 用来对一个元素序列进行重新排序（随机的）

解释：调用 STL 的 `random_shuffle(F,S)`，可以直接对地址为 F 到地址为 S 的位置打乱顺序。

3、随机交换

时间复杂度： $O(n)$

做法：首先随机出 1—N 的一个排列，然后对于第 i 位，把它和 `p[rand()%(i-1)+1]` 进行交换。

代码：

头文件：

```
#include<ctime>
```

```
#include<windows.h>
```

```
int t[N];
```

```
void makePerm1( )
```

```
{
```

```
    srand(time(0));
```

```
    for (int i=1;i<=n;i++) t[i]=i;
```

```
    for (int i=2;i<=n;i++) swap(t[i],t[rand()%(i-1)+1]);
```

```
}
```

二、树

1、问题：在 $O(n)$ 内生成一棵树

2、方法：往前连边

对于第 i 个点，我们把它往前连到 $\text{rand}() \% (i-1) + 1$

代码：

```
void makeTree0( )  
  
{  
  
    srand(time(0));  
  
    for (int i=2;i<=n;i++)  
  
        printf("%d %d\n",i,rand()%(i-1)+1);  
  
}
```

证明：树就是无向无环联通图.

现在要证明它是无环的，且整棵树联通.

①当 $n=2$ 时，满足要求，因为 2 必定连到 1

②假设当 $n < k$ 时都满足，证明当 $n=k$ 时亦满足

$\because$ 连向的点 c 满足 $1 < c < k$

$\therefore$ 不会产生回路

又 $\because$ 之前全都连在一起

$\therefore$ 现在也连在一起

综上所述，该算法可以在 $O(n)$ 内构造出树.

3、特殊要求

如果要求 1 不一定是根，怎么办？

我们进行一次 makePerm，往回连的时候把 $t[i]$ 连到 $t[\text{rand}() \% (i-1) + 1]$.

三、图

1、问题：在 $O(m)$ 内生成一张联通图

2、方法：首先建树，然后剩下的边随便

3、特殊要求

要求 1：没有重边

①对于 N 比较大的情况，每次随机，每次生成的边加到 set 或者 Hash 里面

②对于 N 比较小的情况，把所有的边 (u,v) 列出来，用 `random_shuffle()` 随机二元组

要求 2：没有自环：遇到自环就重新来

要求 3：规定起点，终点连通的有向图

①首先 $t[1]=s$ ，随机 $t[2]$ 到 $t[n]$ ，然后把 $t[1], t[2], t[3], \dots, t_i$ 进行连边

②对于后面的点，随机连到前面，方向也进行随机

③最后随便弄一些边，完善这一棵树

```
int t[...], s, t;
```

```
void makeGraph( )
```

```
{
```

```
    for (int i=1; i<=n; i++) t[i]=i;
```

```
random_shuffle(t+1,t+n+1);
```

```
for (int i=1;i<=n;i++) if (t[i]==s) {swap(t[i],t[1]);break;}
```

```
int i;
```

```
for (i=1;i<n;i++)
```

```
{
```

```
    printf("%d -> %d",t[i],t[i+1]);
```

```
    if (t[i+1]==t) break;
```

```
}
```

```
int k;
```

```
for (i++;i<n;i++)
```

```
{
```

```
    k=rand()&1;
```

```
    if (!k)
```

```
        printf("%d -> %d\n",t[i],t[rand()%(i-1)+1]);
```

```
    else printf("%d -> %d\n",t[rand()%(i-1)+1],t[i]);
```

```
}
```

```
}
```

4、字符串

a、问题：在 $O(L)$ 之内生成长度为 L 的字符串

b、方法

```
char s[...]; int len;
```

```
int makeString(void)
```

```
{
```

```
    for (int i=1;i<=len;i++) s[i]=rand()%M+N;
```

```
    //M 为个数，N 为起始字符
```

```
}
```